

HDOC - Matrix Associative Properties

Menu-Driven C Program Using Separate Functions and Variable-Length 2D Arrays

Objective

Verify the associative property of matrix addition and matrix multiplication by displaying the matrices obtained on both sides of each equation.

Operations

1. Read A, B and C of size $m \times n$ and verify $(A+B)+C = A+(B+C)$.
2. Read A ($m \times n$), B ($n \times p$), C ($p \times q$) and verify $(A \times B) \times C = A \times (B \times C)$.
3. Exit.

Algorithm

Main Program

1. Start and repeatedly display the menu until Exit is selected.
2. For addition, read m and n , create three variable-length matrices A , B and C , and input their elements.
3. Compute $A+B$ and then $(A+B)+C$. Separately compute $B+C$ and then $A+(B+C)$. Display both final matrices and compare corresponding elements.
4. For multiplication, read m , n , p and q . Create A ($m \times n$), B ($n \times p$) and C ($p \times q$), and input their elements.
5. Compute AB , then $(AB)C$. Separately compute BC , then $A(BC)$. Display both final $m \times q$ matrices and compare corresponding elements.
6. Print whether the associative property is verified. For Exit, terminate the program.

C Program

The program uses C99 variable-length arrays (VLAs) and separate functions for input, display, addition, multiplication, comparison, and verification.

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c], char name) {
    int i, j;
    printf("Enter elements of matrix %c (%d x %d):\n", name, r, c);
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void displayMatrix(int r, int c, int a[r][c]) {
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) printf("%6d", a[i][j]);
        printf("\n");
    }
}

void addMatrix(int r, int c, int a[r][c], int b[r][c], int sum[r][c]) {
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];
}

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1], int b[c1][c2], int product[r1][c2]) {
    int i, j, k;
    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            product[i][j] = 0;
            for (k = 0; k < c1; k++)
                product[i][j] += a[i][k] * b[k][j];
        }
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c]) {
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

void verifyAddition(void) {
    int m, n;
    printf("Enter m and n: ");
    scanf("%d%d", &m, &n);
    if (m <= 0 || n <= 0) { printf("Invalid dimensions.\n"); return; }

    int A[m][n], B[m][n], C[m][n];
    int AB[m][n], BC[m][n], left[m][n], right[m][n];
    readMatrix(m, n, A, 'A');
    readMatrix(m, n, B, 'B');
    readMatrix(m, n, C, 'C');

    addMatrix(m, n, A, B, AB);
    addMatrix(m, n, AB, C, left);
    addMatrix(m, n, B, C, BC);
    addMatrix(m, n, A, BC, right);

    printf("\n(A+B)+C =\n"); displayMatrix(m, n, left);
    printf("\nA+(B+C) =\n"); displayMatrix(m, n, right);
    printf("\nAssociative property of addition is %s.\n",
           equalMatrix(m, n, left, right) ? "VERIFIED" : "NOT VERIFIED");
}

void verifyMultiplication(void) {
    int m, n, p, q;
    printf("Enter m, n, p and q: ");
    scanf("%d%d%d%d", &m, &n, &p, &q);
    if (m <= 0 || n <= 0 || p <= 0 || q <= 0) {
        printf("Invalid dimensions.\n"); return;
    }

    int A[m][n], B[n][p], C[p][q];
    int AB[m][p], BC[n][q], left[m][q], right[m][q];
    readMatrix(m, n, A, 'A');
    readMatrix(n, p, B, 'B');
    readMatrix(p, q, C, 'C');

    multiplyMatrix(m, n, p, A, B, AB);
    multiplyMatrix(m, p, q, AB, C, left);
    multiplyMatrix(n, p, q, B, C, BC);
}
```

```

multiplyMatrix(m, n, q, A, BC, right);

printf("\n(A x B) x C =\n"); displayMatrix(m, q, left);
printf("\nA x (B x C) =\n"); displayMatrix(m, q, right);
printf("\nAssociative property of multiplication is %s.\n",
    equalMatrix(m, q, left, right) ? "VERIFIED" : "NOT VERIFIED");
}

int main(void) {
    int choice;
    do {
        printf("\n--- MATRIX ASSOCIATIVE PROPERTY MENU ---\n");
        printf("1. Verify associative property of matrix addition\n");
        printf("2. Verify associative property of matrix multiplication\n");
        printf("3. Exit\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: verifyAddition(); break;
            case 2: verifyMultiplication(); break;
            case 3: printf("Program terminated.\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 3);
    return 0;
}

```

Sample Execution

```
--- MATRIX ASSOCIATIVE PROPERTY MENU ---
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter m and n: 2 2
Enter elements of matrix A (2 x 2):
1 2
3 4
Enter elements of matrix B (2 x 2):
5 6
7 8
Enter elements of matrix C (2 x 2):
1 1
1 1

(A+B)+C =
  7    9
11   13

A+(B+C) =
  7    9
11   13

Associative property of addition is VERIFIED.

Enter your choice: 2
Enter m, n, p and q: 2 2 2 2
A = 1 2 / 3 4
B = 5 6 / 7 8
C = 1 0 / 0 1

(A x B) x C =
 19   22
43   50

A x (B x C) =
 19   22
43   50

Associative property of multiplication is VERIFIED.
```

Conclusion

For matrices with compatible dimensions, both calculations on each side produce identical matrices. Hence matrix addition and matrix multiplication satisfy the associative property.

Compilation note: Because variable-length arrays are used, compile in C99 or later mode, for example: `gcc -std=c99 program.c -o program`.